

第十七届智能车】智能车图像处理（3）-元素识别（十字）

本博客使用的图像是 188*120 的大津法二值化图像。摄像头安装高度为 25cm（离地），前瞻长度约 1m。

在对图像进行最简单的预处理和赛道边界提取后，就可以开始进行元素识别了，这里先从最简单的元素十字开始讲。

十字的情况分为正入十字和斜入十字两种情况，根据视野中十字角点的数量也分为几种情况，要分别进行讨论。

十字的图像特征

在编写程序时，以下各行条件是层层递进进行判定的，一旦有一个条件判定不满足就退出此次判定。

预识别条件：

图像两边都有较多行的边线丢失；

根据实际前瞻需要，选定的两条参考横线，有至少一条两侧均有丢边情况发生；

图像中线几乎为全白；

图像中心存在大面积的白色。

决定识别条件：

十字远端（图像上方）的两个 L 角点均存在；

十字近端（图像下方）的两个 L 角点至少一个存在（考虑到出十字时的斜出情况）。

正入十字示意图：

在这里插入图片描述

斜入十字示意图：

在这里插入图片描述

十字的各个状态

以下状态按照时间顺序排列，且在识别程序的状态切换过程中，进入下一个状态的前提是状态位已经置为上一个状态。

尚未进入十字：视野内远处两个 L 角点存在，此时认为已经识别到十字，十字标志位置位，十字状态位置 1；

车头即将进入十字：视野内仅剩 2 个 L 角点；

车头完全进入十字：视野内没有 L 角点，此时在十字内循迹，十字标志位清除，十字状态位置 2；

即将出十字：车头斜对十字，视野内远处两个 L 角点存在，十字状态位置 3；

即将出十字：视野内仅剩 2 个 L 角点；

出十字：视野内没有 L 角点，清除十字状态位和十字标志位。

各个状态下的图像这里就不一一列举了（我没有保存原始图像，自己画示意图太累了>_<），大家调试的时候可以在板子上接一块屏幕或者使用图传进行观察，基本上就是我说的这几个状态。

识别程序编写思路

需要提前设置的一些阈值- 十字两侧边线丢线阈值，只有在两侧丢线行数大于此阈值后才进行十字判定- 十字判定中心区域存在大量白色像素数量总和阈值- 十字判定中心存在大量白色的矩形区域四个顶点坐标- 十字上、下 L 角点容差阈值，用于判定上下 L 角点的存在性- 十字补线最小二乘法从角点上溯行数阈值，用于补线

思路（1）检测图像两侧下方的丢线；（2）检测图像中线上的黑色部分是否很长，如果很长就终止判断（防止在三岔误判）；（3）检测选定的两行参考线是否至少有一根两侧丢线（用于斜入十字的辅助判定）；（4）检测图像中心选定矩形区域内是否存在大量白点（防止与三岔、弯道误判）；（5）L 角点的判定①预扫线：从图像中线开始向上方扫描黑白交界点，采用每隔 10 像素扫线法提高效率，若未扫到，扫描最上方 10 个像素以防漏扫；从图像中线向下方扫线，因为下方拐点存在纵向上小于 10 像素的尖，而且在同一列中会有两次黑白交界，所以采用逐像素向下扫描的方式以防漏扫以及误判；将两次扫描得到的上下边界分别存进两个数组内。②L 型角点判定：遍历黑白交界点纵坐标数组，从左向右和从右向左两个方向扫描。a)找上方 L 型角点：寻找黑白交界点纵坐标开始变小的点，这一步不推荐使用边线斜率的变化进行判定，因为图像锯齿的存在，很容易出现误判的情况；b)识别到十字后，开始寻找十字下方 L 型角点：检查下方黑白交界数组中是否存在前几个交界纵坐标还是接近的，但是再过去就发生突变，这时候就可以认为找到了下方 L 角点。在这里插入图片描述（6）上方两个 L 角点存在，则认为判定到了十字，标志位置位。

补线程序编写思路

注意 1：补线需要直接补进边线数组，如果只是计算了直线的 k 和 b，欲图在图像上补线的话，因为计算出来的数据不一定是整数，即使进行了强制类型转换，也会因为四舍五入导致好几个点计算出来的坐标一致，很容易出现补线截断的情况！

注意 2：补线需要限制斜率绝对值的范围，十字的补线相对陡峭，斜率不可能很小，如果对斜率不做限制，补出来的线就会发生严重且致命的抖动！

状态 1：分别连接左右两侧的上下 L 角点。

状态 2：从左右两侧的 L 型角点开始，向上方上溯若干个像素，用最小二乘法拟合这一段边线得到一条直线，再往下补线进入边线数组即可。

状态 4：出十字车头斜对，连接上下 L 角点均存在的一侧的上下 L 角点，另一侧根据上 L 角点上溯最小二乘法拟合直线进行补线。

状态 5：同状态 2。

总结

十字在所有元素中相对简单，因为即使不做识别和补线，仅仅依赖循迹也是可以通过的，但是为了防止其他元素在十字中的误判，还是需要对十字进行识别，锁定在十字中的状态；十字的特征主要就是四个 90°的 L 形角点，抓住特征识别，误判率很低而且效果不错；

十字的补线也是相对简单的，就是需要根据四个角点的存在性分别进行不同的补线，一定要分清楚具体的情况，还有一定要注意补线斜率的限幅，可以先测一测直道上正常的边线斜率大概是多少，然后设定一个合适的阈值进行限制，防止抖动。